

Graph Style Specification

Declarative palettes, node and edge styles, rules, labels, and presets

mdstats

Contents

Purpose and status	1
Motive	1
Normative ownership	1
AI context summary	2
Chemical color palette	2
ChemicalColorPalette	2
Style data model	3
NodeStyle	3
NodeStylePatch	3
EdgeStyle	3
EdgeStylePatch	3
Style rules	3
GraphLabelOptions	4
Global node-display mode	4
NodeDisplayMode	4
GraphStyle	5
Rule-resolution algorithm	6
Endpoint color modes	6
Validation rules	6
Preset responsibilities	6
Edge cases and cautions	6
Testing requirements	7
Public exports	7
Future compatibility	7

Purpose and status

This document is the normative specification for `mdstats.plotting.graph_styles`. It defines renderer-shared visual encodings for graph nodes and edges.

A style is declarative presentation data. It must not alter graph selection, layout, scientific metadata, connectivity, or identity.

Motive

Scientific adapters expose metadata such as species, degree, component, bridge type, ring size, and transition status. A shared style system maps those attributes into consistent visual encodings without hard-coding scientific meanings into renderers.

Normative ownership

This specification owns:

- chemical color palettes;
- complete node and edge styles;
- partial style patches;
- metadata rules and precedence;
- label requests;
- shared graph-style presets;
- global node-display modes;
- style validation and resolution semantics.

Renderer-specific artist batching, line segmentation, and legend construction belong to the renderer specification.

AI context summary

- Styles affect appearance only.
- Rules match exact metadata values; arbitrary callbacks are deferred.
- Defaults are applied first, then matching patches in deterministic priority order.
- Later matching rules may override fields changed by earlier rules.
- A missing referenced metadata column is an error.
- Chemical colors are conventions, not scientific data.
- Labels are off unless explicitly requested.
- Node markers may be rendered normally, reduced to compact color-coded dots, or hidden while retaining the scientific node set.

Chemical color palette

ChemicalColorPalette

```
@dataclass(frozen=True, slots=True)
class ChemicalColorPalette:
    colors: Mapping[str, ColorLike]
    fallback_color: ColorLike = "#808080"
    name: str = "mdstats-default"

    @classmethod
    def default(cls) -> "ChemicalColorPalette": ...

    @classmethod
    def monochrome(cls) -> "ChemicalColorPalette": ...

    def with_overrides(
        self,
        overrides: Mapping[str, ColorLike],
    ) -> "ChemicalColorPalette": ...
```

The default palette is a visualization convention, not scientific data. The initial palette should include at least:

Species	Default color
H	white
C	dark gray
N	blue-violet
O	red
F	light green
Si	blue
Al	cyan
P	orange
S	yellow
Cl	green
Li	light violet
Na	violet
K	dark violet

Every color must be accepted by Matplotlib. User overrides take precedence.

Style data model

NodeStyle

```
@dataclass(frozen=True, slots=True)
class NodeStyle:
    face_color: ColorLike = "#808080"
    size: float = 36.0
    marker: str = "o"
    alpha: float = 1.0
    edge_color: ColorLike = "#202020"
    edge_width: float = 0.75
    zorder: float = 3.0
```

size is in points squared, matching Matplotlib scatter. Widths and sizes must be nonnegative. Alpha must lie in $[0, 1]$.

NodeStylePatch

```
@dataclass(frozen=True, slots=True)
class NodeStylePatch:
    face_color: ColorLike | None = None
    size: float | None = None
    marker: str | None = None
    alpha: float | None = None
    edge_color: ColorLike | None = None
    edge_width: float | None = None
    zorder: float | None = None
```

A patch changes only non-None fields.

EdgeStyle

```
@dataclass(frozen=True, slots=True)
class EdgeStyle:
    color: ColorLike = "#707070"
    width: float = 1.2
    alpha: float = 0.65
    line_style: Literal[
        "solid", "dashed", "dotted", "dashdot"
    ] = "solid"
    color_mode: Literal[
        "constant",
        "source",
        "target",
        "midpoint_split",
        "segmented_gradient",
    ] = "constant"
    gradient_segments: int = 8
    zorder: float = 1.0
```

gradient_segments is used only for segmented gradients and must be at least 2.

EdgeStylePatch

EdgeStylePatch mirrors EdgeStyle with optional fields.

Style rules

```
@dataclass(frozen=True, slots=True)
class NodeStyleRule:
    attribute: str
    values: tuple[AttributeScalar, ...]
    patch: NodeStylePatch
    priority: int = 0
```

```
@dataclass(frozen=True, slots=True)
class EdgeStyleRule:
    attribute: str
    values: tuple[AttributeScalar, ...]
    patch: EdgeStylePatch
    priority: int = 0
```

Rule resolution is deterministic:

1. start from the default style;
2. collect all matching rules;
3. sort by ascending priority and declaration order;
4. apply each patch in order;
5. later patches override earlier fields.

Missing rule attributes are errors during style validation. An empty values tuple is invalid.

GraphLabelOptions

```
@dataclass(frozen=True, slots=True)
class GraphLabelOptions:
    node_attribute: str | None = None
    edge_attribute: str | None = None
    node_keys: tuple[GraphKey, ...] | None = None
    edge_keys: tuple[GraphKey, ...] | None = None
    font_size: float = 8.0
    node_offset_points: tuple[float, float] = (3.0, 3.0)
```

Labels are disabled when both attributes are None.

When a label attribute is set:

- node_keys=None means all selected nodes;
- edge_keys=None means all selected edges;
- the complexity policy still limits the final label count.

Labels are off by default. The first renderer does not promise automatic collision-free placement.

Global node-display mode

NodeDisplayMode

```
class NodeDisplayMode(str, Enum):
    MARKERS = "markers"
    DOTS = "dots"
    HIDDEN = "hidden"
```

The node-display mode is renderer-shared presentation state. It does not filter nodes, change graph identity, alter edge endpoints, or modify periodic materialization.

MARKERS Use the fully resolved NodeStyle, including marker shape, size, outline, and alpha.

DOTS Preserve each node's resolved face color and alpha, but render every node as a small circular point with no outline. The shared point area is GraphStyle.node_dot_size. Species and role color coding therefore remain visible while the graph occupies less screen area.

HIDDEN Do not create node or periodic-ghost marker artists. Node labels and node legend entries are suppressed. Scientific node keys, node positions, edge endpoint indices, edge colors derived from endpoint metadata, and hover-independent source mappings remain intact.

The mode applies to both canonical nodes and display-only replicas or ghosts. Hidden nodes are a presentation choice, not an omission from the graph.

GraphStyle

```
@dataclass(frozen=True, slots=True)
class GraphStyle:
    node_default: NodeStyle = field(default_factory=NodeStyle)
    edge_default: EdgeStyle = field(default_factory=EdgeStyle)
    node_rules: tuple[NodeStyleRule, ...] = ()
    edge_rules: tuple[EdgeStyleRule, ...] = ()
    labels: GraphLabelOptions = field(
        default_factory=GraphLabelOptions
    )
    palette: ChemicalColorPalette = field(
        default_factory=ChemicalColorPalette.default
    )
    background_color: ColorLike = "white"
    legend: Literal["auto", "none", "all"] = "auto"
    node_display_mode: NodeDisplayMode | str = NodeDisplayMode.MARKERS
    node_dot_size: float = 16.0

    @classmethod
    def default(cls) -> "GraphStyle": ...

    @classmethod
    def atomic_default(
        cls,
        palette: ChemicalColorPalette | None = None,
    ) -> "GraphStyle": ...

    @classmethod
    def framework_default(
        cls,
        palette: ChemicalColorPalette | None = None,
        *,
        diagnostic: bool = False,
        node_display_mode: NodeDisplayMode | str = NodeDisplayMode.MARKERS,
        node_dot_size: float = 16.0,
    ) -> "GraphStyle": ...

    @classmethod
    def transition_default(
        cls,
        palette: ChemicalColorPalette | None = None,
    ) -> "GraphStyle": ...

    @classmethod
    def publication(cls) -> "GraphStyle": ...
```

`atomic_default()` colors nodes by the node attribute symbol and uses endpoint-colored edges.

`framework_default()` consumes the adapter-provided node attributes symbol and `framework_role` and the edge attribute `edge_kind`. In projected mode it uses constant bridge-kind colors and a wider framework edge. With `diagnostic=True`, linker nodes are smaller and retained atomic-path segments use endpoint-split colors. The preset maps common edge kinds including `direct`, `oxygen_bridge`, `sulfur_bridge`, and double-linker variants; unknown kinds retain the neutral default. The factory accepts `node_display_mode` and `node_dot_size` directly for compact or edge-only framework views.

`transition_default()` uses the edge attribute `transition_status`:

unchanged	light gray, thin, partially transparent
removed	red, dashed, emphasized
added	green, solid, emphasized

It uses the node attribute `affected` to add a dark outline or larger marker.

Rule-resolution algorithm

For each node or edge:

1. start from the corresponding complete default style;
2. sort rules by (priority, declaration_order) in ascending order;
3. test exact membership of the metadata value in the rule's values tuple;
4. apply each matching patch in sorted order;
5. return the final immutable complete style.

This means a higher-priority rule is applied later and can override a lower-priority rule. Ties retain declaration order.

Endpoint color modes

The `EdgeStyle.color_mode` field has the following renderer-independent meanings:

- constant: use `EdgeStyle.color` for the complete edge;
- source: use the rendered source-node face color;
- target: use the rendered target-node face color;
- midpoint_split: color the first and second halves by source and target colors;
- segmented_gradient: interpolate source to target color across a fixed number of segments.

A backend may implement these modes differently at the artist level, but it must preserve their visual meaning. The segmented mode is complexity-accounted because it can multiply artist segments.

Validation rules

Colors must be valid backend-compatible color values under the current Matplotlib implementation. Sizes and line widths must be finite and nonnegative. `node_dot_size` must be finite and strictly positive. `node_display_mode` must resolve to one of the three declared enum values. Alpha values must lie in `[0, 1]`. Markers must be valid nonempty Matplotlib marker strings. `gradient_segments` must be an integer of at least two.

Rules and label options must reference attributes present in the prepared graph view. Unknown attributes raise `GraphStyleError` before artist creation.

Preset responsibilities

`GraphStyle.atomic_default()` colors nodes by the `symbol` attribute and uses endpoint-split edges. `GraphStyle.framework_default()` supplies a shared visual preset for the metadata contract owned by the framework-topology adapter; it does not infer roles or bridge kinds. `GraphStyle.transition_default()` emphasizes affected nodes, added edges, and removed edges. `GraphStyle.publication()` supplies restrained generic sizes and widths but does not infer scientific classes.

Future ring presets may be added when a ring adapter defines a stable metadata contract. The style module may map declared metadata to appearance, but it must never compute framework or ring science.

Edge cases and cautions

- A white hydrogen node may disappear on a white background without a dark outline.
- Excessively large node sizes can hide short edges.
- DOTS intentionally discards marker shape and outline distinctions; do not use it when those encodings are scientifically important.
- HIDDEN removes node hover targets and node legends, so edge metadata must carry enough context for interpretation.
- Hidden nodes still count toward graph and periodic complexity because they remain part of the authoritative display graph.
- Segmented gradients can create very large vector files.
- Multiple rules matching the same item are intentional; precedence must be explicit.
- Color alone should not be the only encoding for critical diagnostic differences; line style or width should also be used.
- Palette overrides must return a new immutable palette.

Testing requirements

Tests must cover:

- validation of colors, markers, alpha, widths, and segment counts;
- deep immutability of palette mappings;
- deterministic rule precedence;
- exact matching of scalar and tuple-valued attributes;
- validation and normalization of all node-display modes;
- preservation of resolved node colors in DOTS mode;
- suppression of node artists, ghosts, labels, legends, and 3-D node traces in HIDDEN mode;
- missing-attribute failures;
- label-key and label-attribute validation;
- atomic, framework, diagnostic-framework, and transition preset behavior;
- renderer reuse of resolved styles without modification.

Public exports

ChemicalColorPalette

NodeStyle

NodeStylePatch

EdgeStyle

EdgeStylePatch

NodeStyleRule

EdgeStyleRule

GraphLabelOptions

GraphStyle

The helper functions that validate and resolve styles are implementation contracts and are not part of the root public API.

Future compatibility

The same GraphStyle contract should be consumed by the future Plotly backend. Renderer-specific options such as hover templates, camera settings, and WebGL trace choices must not be added to GraphStyle.